Formação Completa

AWS Developer Associate

Interagindo com AWS Services via SDK e APIs





O que vamos abordar

1 **Utilização de SDKs AWS**

Como os SDKs facilitam a interação programática com serviços AWS.

2 **Boto3 para Python**

A biblioteca padrão para desenvolvimento de aplicações AWS em Python.

3 **Configuração e Segurança**

Gerenciamento de credenciais e tratamento de exceções.

4 **Exemplos**

Demonstrações de código para interagir com serviços como S3.

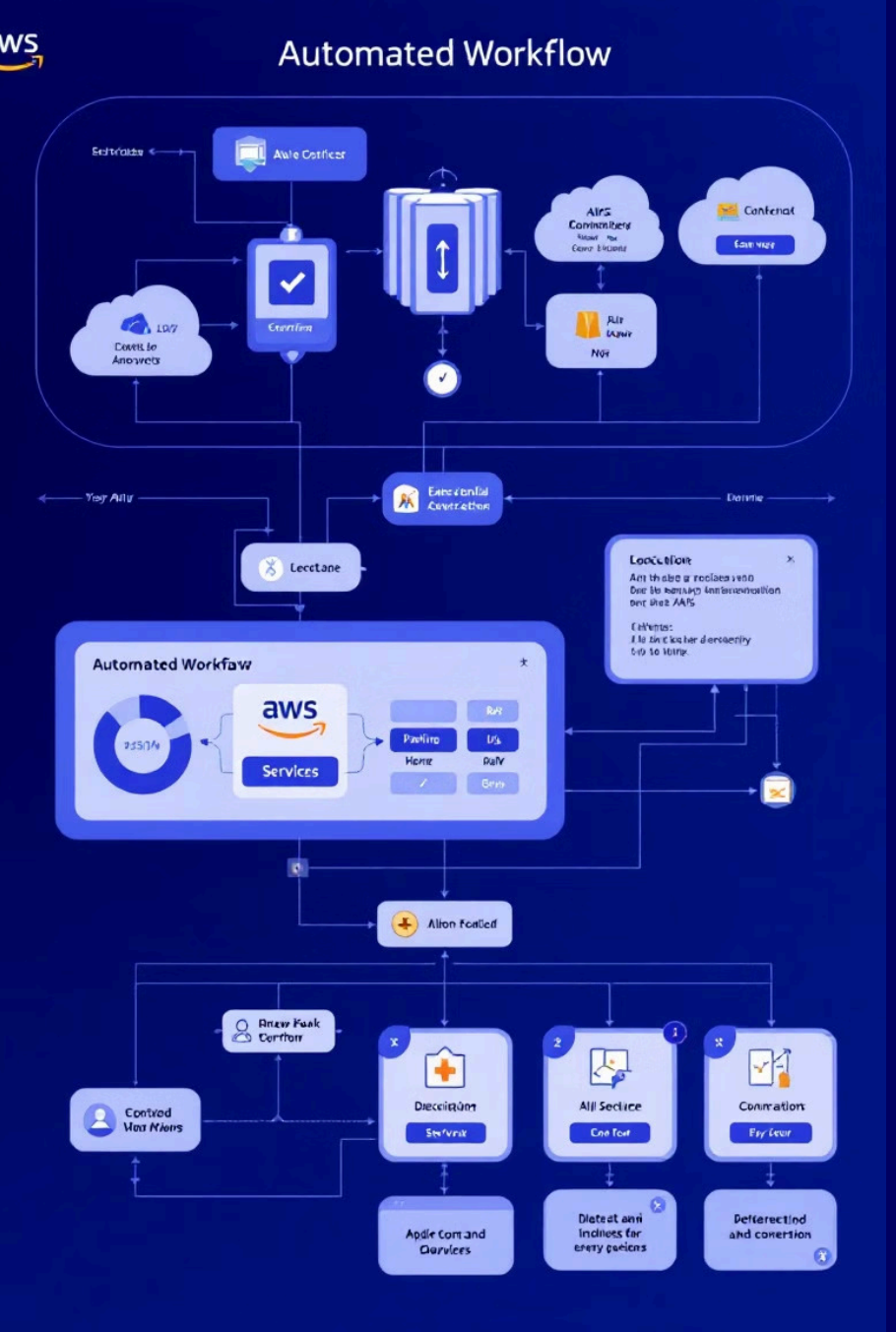
Introdução ao Boto3

O que é Boto3?

Biblioteca oficial da AWS para Python que permite interação programática com serviços AWS.

Principais recursos

Suporte a diversos serviços como S3, DynamoDB, SQS e muitos outros.



Vantagens do Boto3

Automação

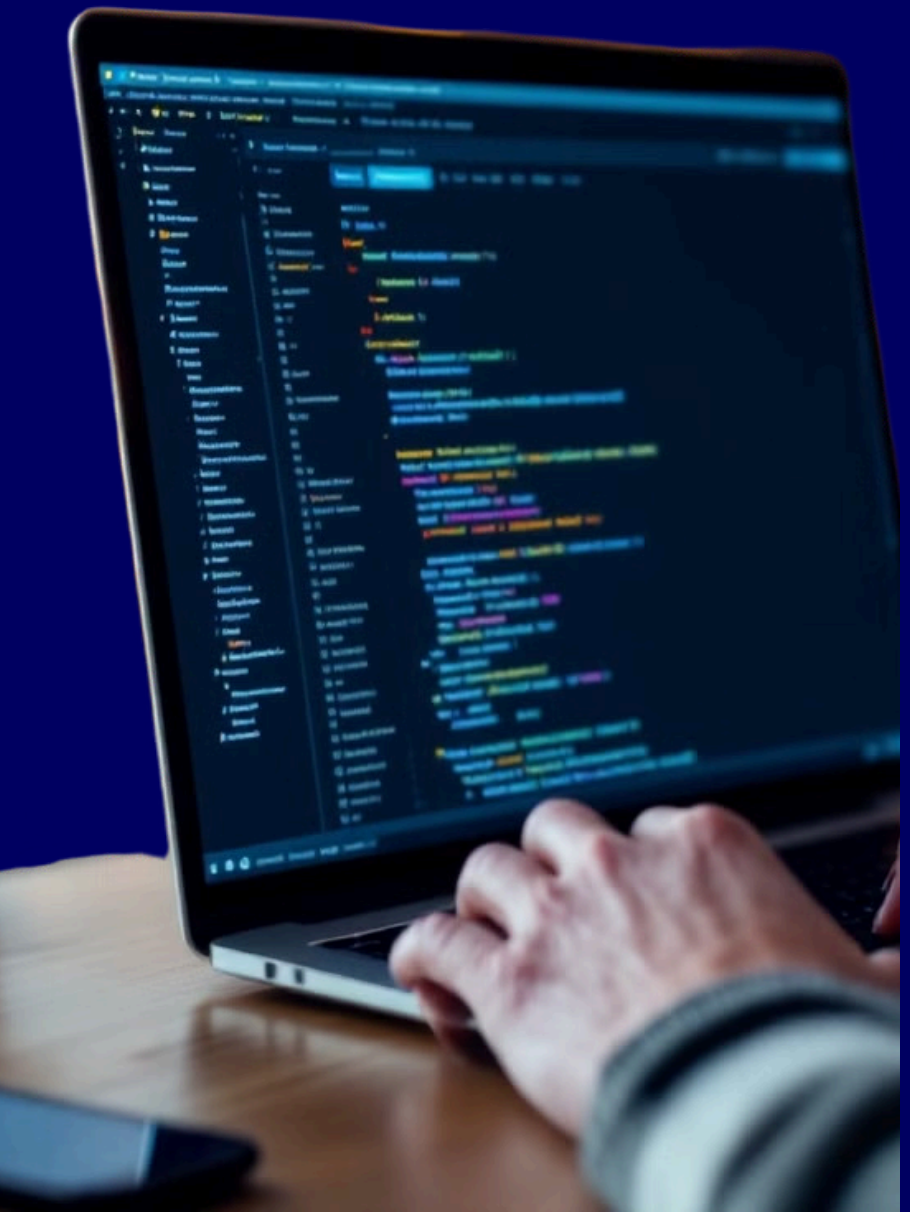
Permite automatizar tarefas repetitivas e criar fluxos de trabalho complexos.

Integração

Facilita a comunicação entre diferentes serviços AWS em uma única aplicação.

Robustez

Oferece mecanismos para tratamento de erros e retry de operações.



Configuração Básica do Boto3

1

Instalação

Instale o Boto3 via pip: pip install boto3

2

Importação

Importe a biblioteca em seu código: import boto3

3

Configuração

Configure uma sessão com região: session = boto3.Session(region_name='us-east-1')

Configuração de Sessão



Região

Defina a região correta para direcionar operações aos recursos apropriados.



Credenciais

Configure credenciais seguras para autenticar todas as operações.



Segurança

Uma sessão bem configurada é a base para inter

Configuração de Sessão: Métodos recomendados

AWS CLI (Alternativa Segura)

Configure credenciais de forma segura através do terminal.

aws configure

Insira Access Key, Secret Key e região quando solicitado.

- **AWS Access Key ID**
- **AWS Secret Access Key**
- **Região padrão** (ex: us-east-1)
- **Formato de saída** (pode deixar json)

Armazena (localmente) credenciais em ~/.aws/credentials (Linux/Mac) ou C:\Users\USUÁRIO\.aws\credentials (Windows).

Configuração de Sessão: Métodos recomendados

Variáveis de Ambiente (Acesso temporário)

Alternativa sem armazenamento em arquivos locais.

Linux/Mac:

```
export AWS_ACCESS_KEY_ID="SUA_KEY"

export AWS_SECRET_ACCESS_KEY="SUA_SECRET"

export AWS_DEFAULT_REGION="us-east-1"
```

No Windows (cmd/prompt):

```
set AWS_ACCESS_KEY_ID=SUA_ACCESS_KEY

set AWS_SECRET_ACCESS_KEY=SUA_SECRET_KEY

set AWS_DEFAULT_REGION=us-east-1
```

Windows (PowerShell):

```
$env:AWS_ACCESS_KEY_ID="SUA_ACCESS_KEY"

$env:AWS_SECRET_ACCESS_KEY="SUA_SECRET_KEY"

$env:AWS_DEFAULT_REGION="us-east-1"
```

Configuração de Sessão: Métodos recomendados

Usar o Arquivo .env (Se estiver usando Python em um projeto)

Instale a biblioteca:

```
pip install python-dotenv
```

Crie um arquivo .env e adicione as credenciais:

```
AWS_ACCESS_KEY_ID=SUA_ACCESS_KEY

AWS_SECRET_ACCESS_KEY=SUA_SECRET_KEY

AWS_DEFAULT_REGION=us-east-1
```

A partir de agora, você pode carregar as suas credenciais no seu código Python.

Vamos para o exemplo!

Exemplo Básico: Listar Buckets S3 carregando as credenciais do .env

```python
import boto3
import os
from dotenv import load_dotenv

# Carrega as variáveis do arquivo .env
load_dotenv()

# Configuração de Sessão
session = boto3.Session(
    aws_access_key_id=os.getenv("AWS_ACCESS_KEY_ID"),
    aws_secret_access_key=os.getenv("AWS_SECRET_ACCESS_KEY"),
    region_name=os.getenv("AWS_REGION")
)

# Criar cliente S3
s3_client = session.client("s3")

# Listar Buckets
response = s3_client.list_buckets()
buckets = [bucket["Name"] for bucket in response.get("Buckets", [])]
print("Buckets disponíveis:", buckets)
```

Importância da Configuração Correta

Precisão

Direciona operações aos recursos corretos.

Integração

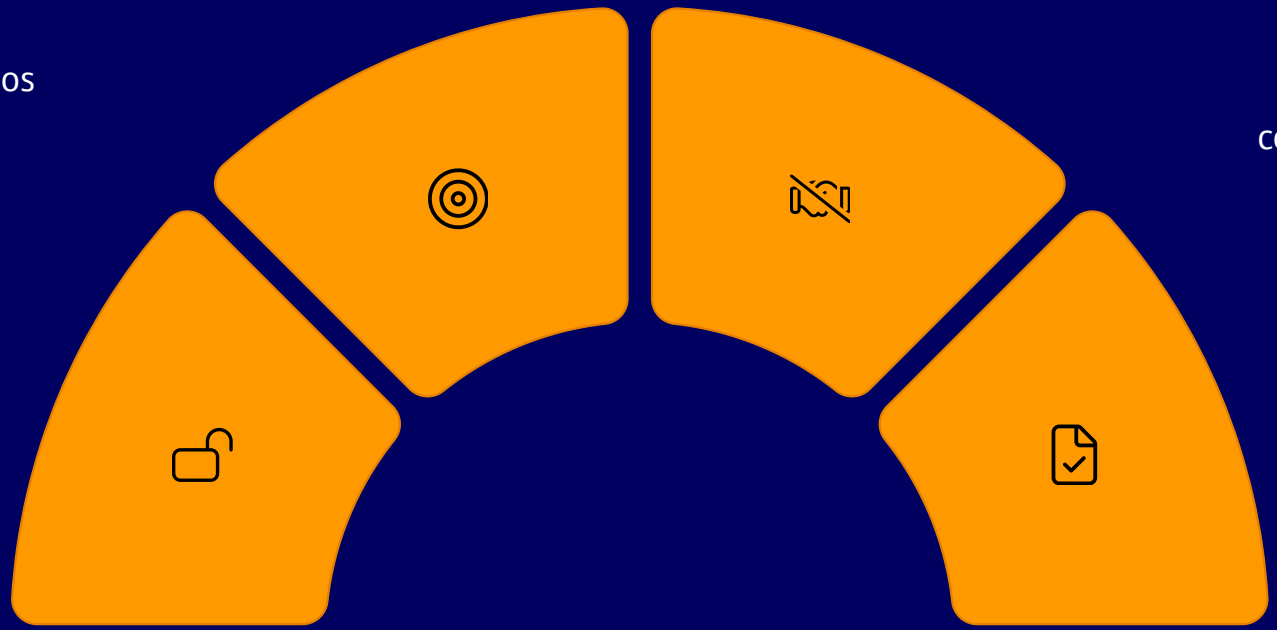
Facilita a integração de múltiplos serviços.

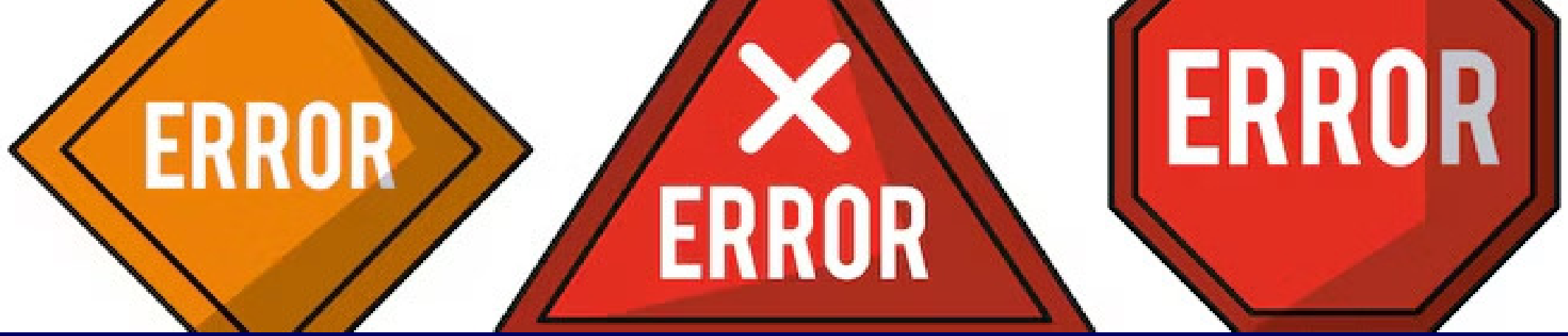
Segurança

Garante acesso seguro aos serviços AWS.

Consistência

Assegura interações consistentes com a AWS.





Tratamento de Erros

Identificar Exceções

Conheça as exceções comuns como ResourceNotFound e AccessDenied.

Implementar Try/Except

Use blocos try/except para capturar e tratar erros adequadamente.

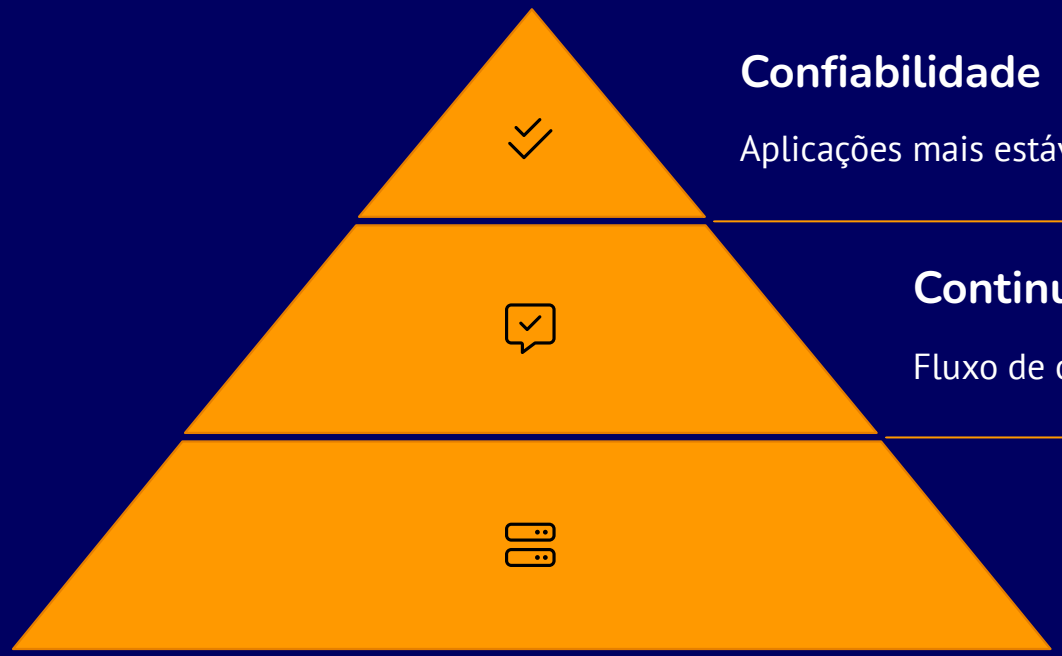
Estratégias de Retry

Implemente lógicas de retry para falhas temporárias.

Fallback

Crie mecanismos de fallback para manter a continuidade do serviço.

Benefícios do Tratamento de Erros



Confiabilidade

Aplicações mais estáveis e confiáveis

Continuidade

Fluxo de operações sem interrupções

Integridade

Preservação da integridade dos dados

Configuração de Sessão e Tratamento de Erros

```python
import boto3
from botocore.exceptions import NoCredentialsError, EndpointConnectionError

try:
    # Configura uma sessão com a região desejada
    session = boto3.Session(region_name='us-east-1')
    s3_client = session.client('s3')

    # Lista os buckets disponíveis
    response = s3_client.list_buckets()
    buckets = [bucket['Name'] for bucket in response.get('Buckets', [])]
    print("Buckets disponíveis:", buckets)

except NoCredentialsError:
    print("Erro: Credenciais não encontradas. Verifique suas configurações de autenticação.")

except EndpointConnectionError:
    print("Erro: Não foi possível conectar ao endpoint. Verifique sua conexão de rede ou a região especificada.")

except Exception as e:
    print(f"Ocorreu um erro inesperado: {str(e)}")
```

Configura uma sessão com o S3, lista os buckets disponíveis e trata erros comuns como falhas de credenciais e conexão.

Exceções Comuns no Boto3

ResourceNotFoundException

Ocorre quando o recurso solicitado não existe na AWS.

AccessDenied

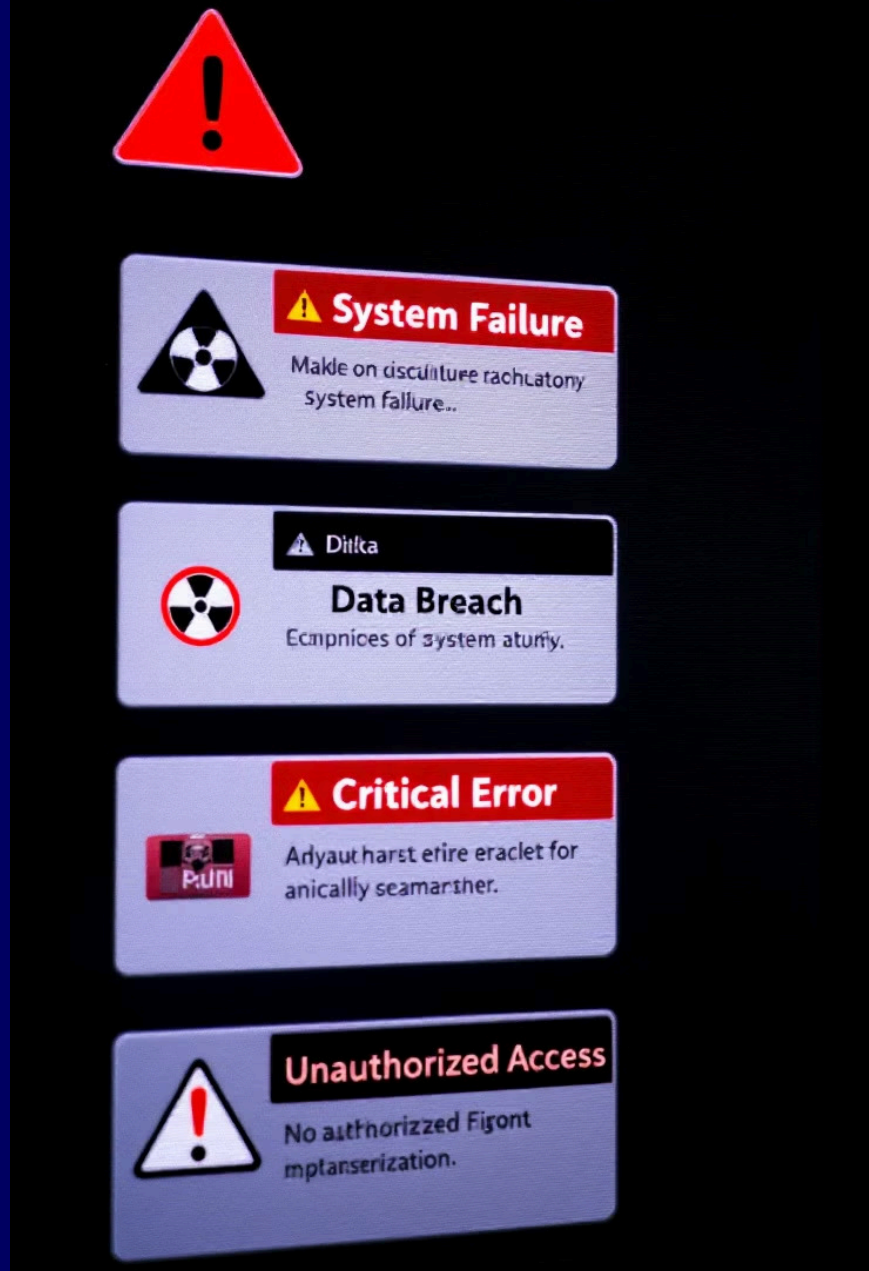
Indica que as credenciais não têm permissão para acessar o recurso.

ValidationError

Acontece quando os parâmetros fornecidos são inválidos.

ConnectionError

Surge quando há problemas de conectividade com os serviços AWS.





AWS Key Management Service (KMS)

O que é?

Serviço da AWS para criação e gerenciamento de chaves criptográficas.

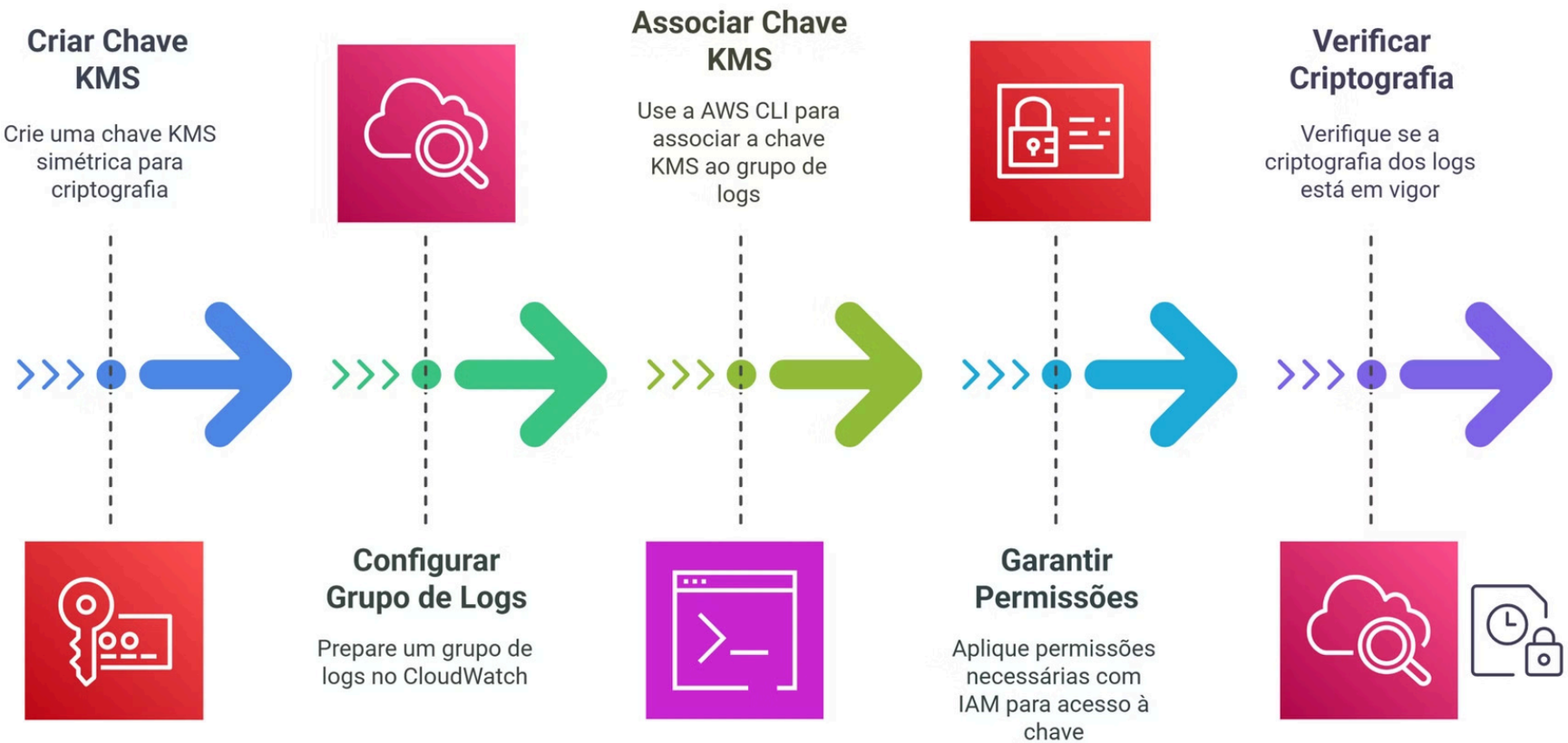
Função

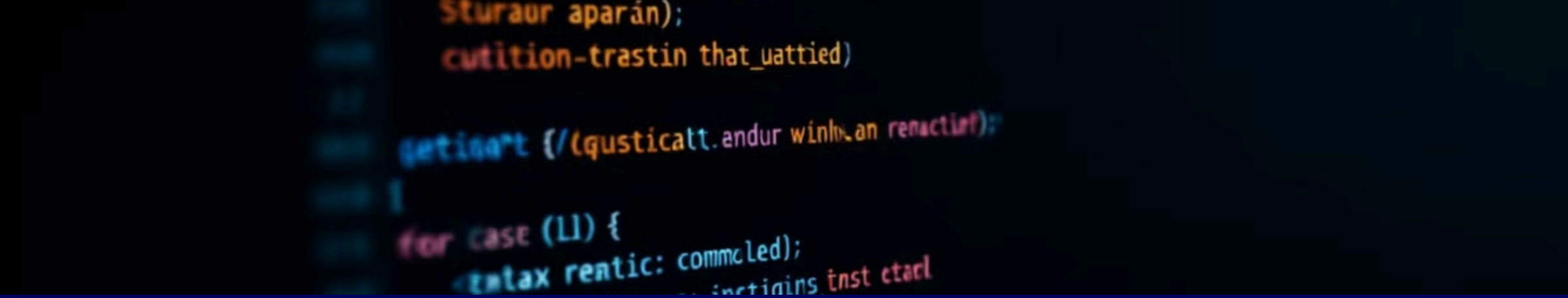
Permite criptografar dados em repouso e controlar o acesso às chaves.

Aplicação

Usado para proteger logs, segredos e outros dados sensíveis na AWS.

Criptografia de Logs com KMS





Comando para Associar KMS a Logs

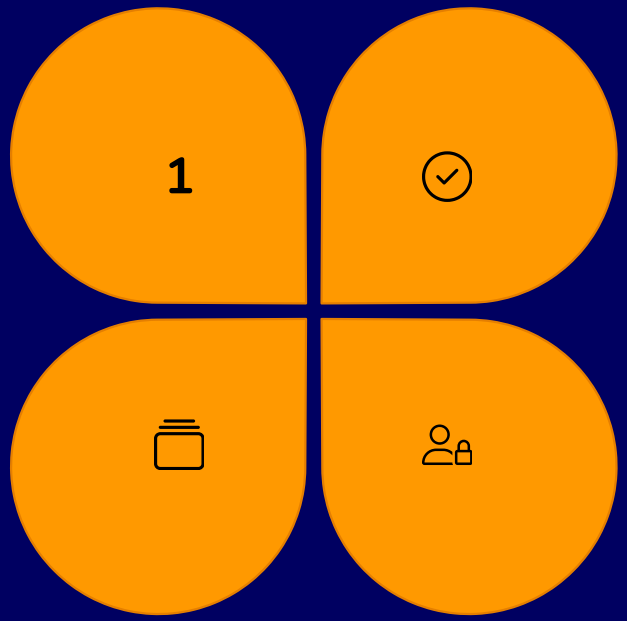
```
aws logs associate-kms-key --log-group-name "/aws/lambda/MinhaFuncaoLambda"
     --kms-key-id "arn:aws:kms:us-east-1:123456789012:key/abcdef12
    -3456-7890-abcd-ef1234567890"
```

Este comando vincula a chave KMS especificada ao grupo de logs, garantindo criptografia em repouso.

Benefícios da Criptografia de Logs

Segurança

Protege informações sensíveis nos logs.



Conformidade

Atende requisitos regulatórios de proteção de dados.

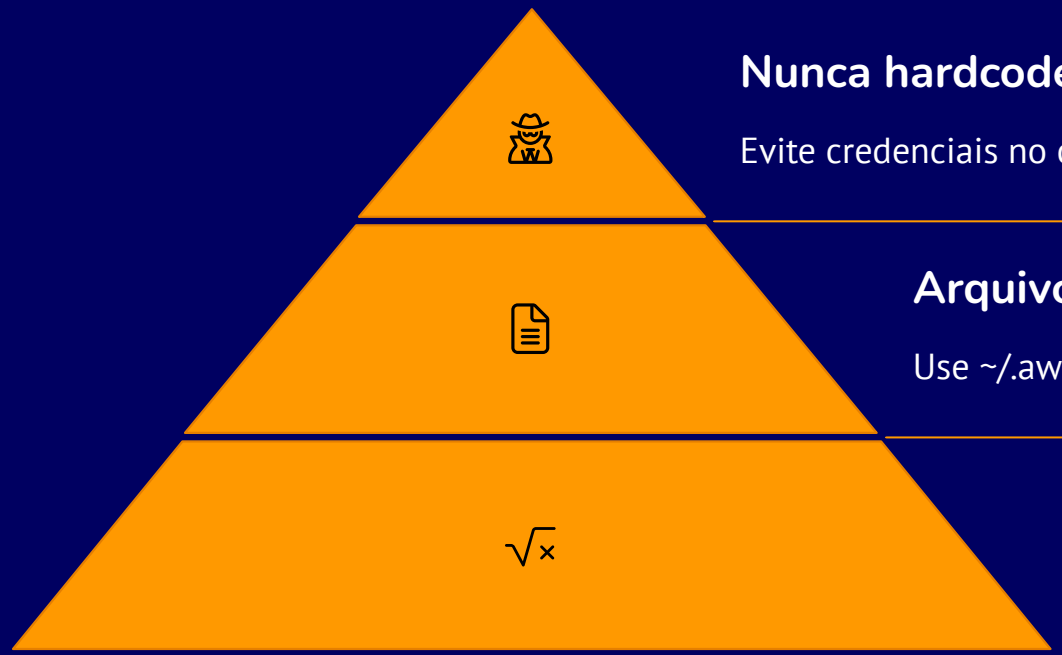
Auditoria

Facilita auditoria de acesso aos dados sensíveis.

Controle de Acesso

Restringe acesso aos logs apenas a usuários autorizados.

Boas Práticas: Gerenciamento de Credenciais



Nunca hardcode

Evite credenciais no código-fonte

Arquivos de configuração

Use ~/.aws/credentials e ~/.aws/config

Variáveis de ambiente

Configure AWS_ACCESS_KEY_ID e AWS_SECRET_ACCESS_KEY



Boas Práticas: Segurança

Princípio do Menor Privilégio

Conceda apenas as permissões necessárias para cada função.

Rotação de Credenciais

Troque regularmente chaves de acesso para reduzir riscos.

Criptografia

Use criptografia para dados sensíveis em trânsito e repouso.

Auditoria

Monitore e registre todas as ações realizadas via SDK.

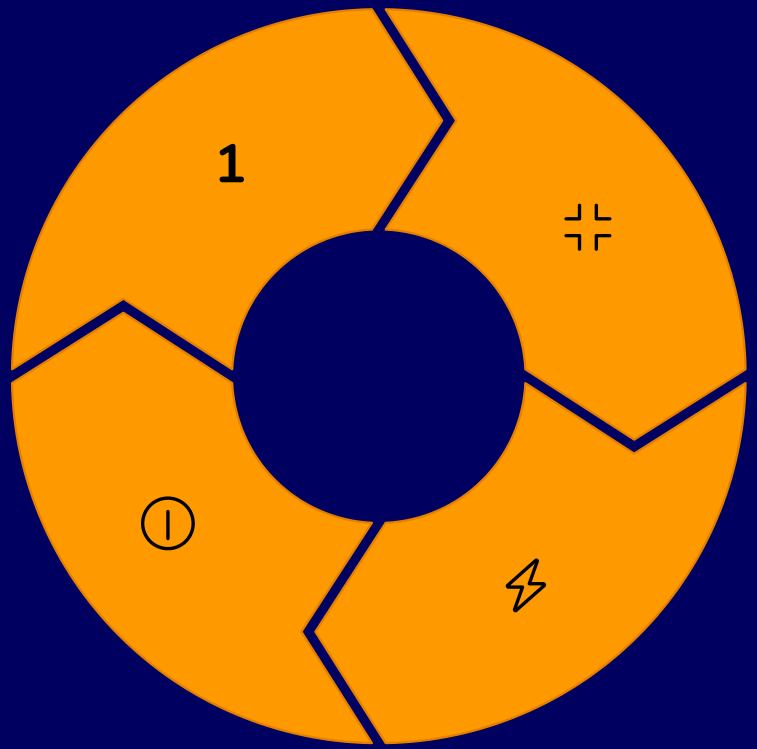
Boas Práticas: Performance

Reutilizar Clientes

Crie clientes uma vez e reutilize-os

Timeout Adequado

Configure timeouts apropriados para cada operação



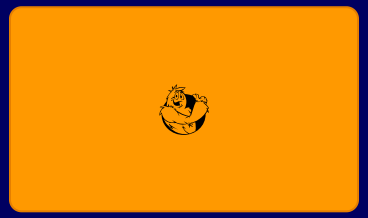
Paginação

Use paginação para grandes conjuntos de dados

Operações em Lote

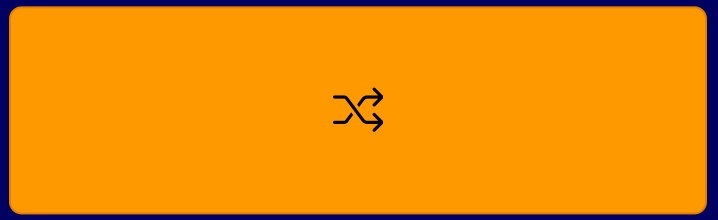
Agrupe operações quando possível

Boas Práticas: Resiliência



Implementar Retries

Use backoff exponencial com jitter



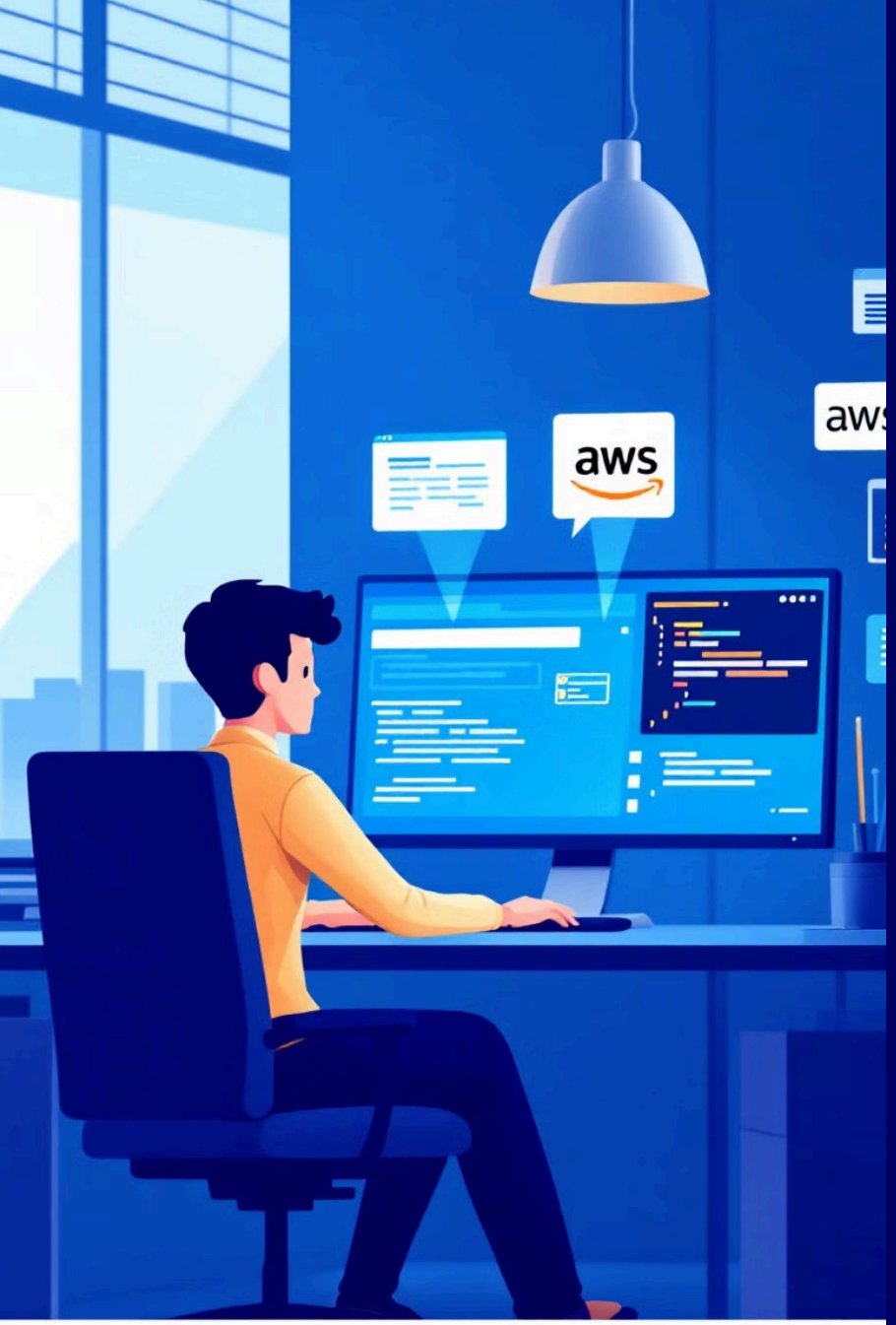
Circuit Breaker

Evite sobrecarregar serviços com falhas



Fallback

Tenha alternativas para serviços críticos



Ferramentas Complementares

AWS CLI

Interface de linha de comando para interagir com serviços AWS.

AWS CDK

Framework para definir infraestrutura como código usando linguagens familiares.

LocalStack

Emulador de serviços AWS para desenvolvimento e testes locais.

Casos de Uso Comuns



Backup Automatizado

Scripts para backup periódico de dados para S3.



Processamento de Dados

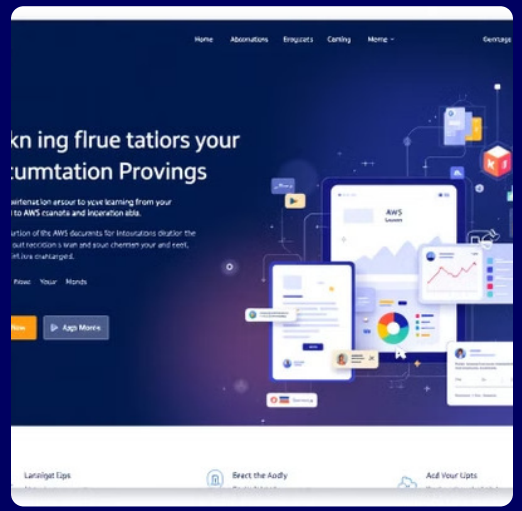
Pipelines de ETL usando Lambda, S3 e outros serviços.



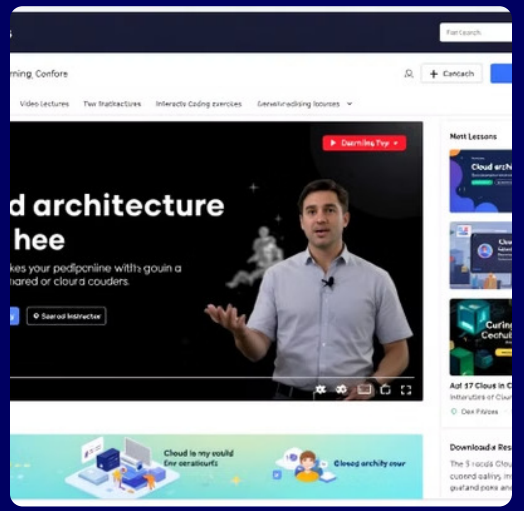
Automação de Infraestrutura

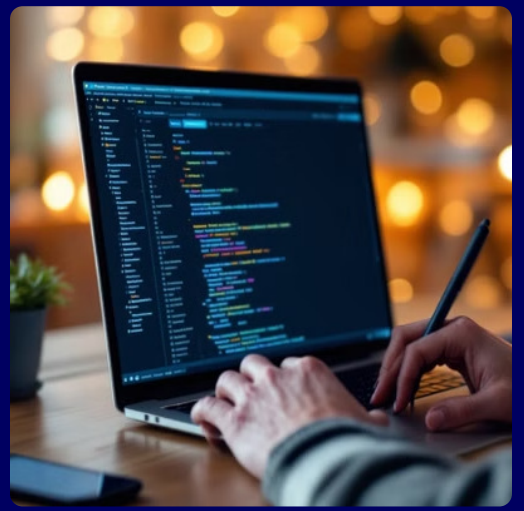
Provisionamento e configuração automatizada de recursos.

Recursos de Aprendizado









Explore a documentação oficial da AWS, cursos online, livros especializados e tutoriais práticos para aprofundar seus conhecimentos.

Tópicos Essenciais para AWS SDK

Relembre

Idempotência

Garante que operações repetidas produzam o mesmo resultado, evitando duplicações indesejadas.

Utilize chaves de idempotência quando disponíveis, especialmente com DynamoDB.

Controle de Duplicação

Verifique a existência prévia de recursos antes de criar novos.

Crucial para sistemas que processam requisições potencialmente repetidas.

Tratamento de Erros HTTP

Interprete corretamente códigos HTTP (200, 400, 404, 500) para decisões adequadas.

Implemente retries para falhas temporárias e fallbacks para erros críticos.

A implementação robusta destes conceitos garante a confiabilidade e integridade dos dados em suas aplicações AWS.

Por que o que vimos sobre Interação com os Serviços AWS via SDK e APIs é Importante?

(Visão Prática e da Certificação)

Interação com AWS via SDK

Certificação AWS Developer

Conhecimento de SDKs é requisito fundamental para automatizar operações na nuvem.

Automação Eficiente

Scripts bem elaborados reduzem erros manuais e permitem escala operacional.

Integração de Serviços

Boto3 facilita a comunicação entre diversos serviços AWS em soluções completas.

Na certificação AWS Developer Associate, é essencial demonstrar domínio prático do Boto3 para criar aplicações robustas e seguras.

A automação via SDK permite construir sistemas resilientes que se comunicam eficientemente com a nuvem, garantindo escalabilidade e manutenção simplificada.

Configuração e Tratamento de Sessões/Erros

- **Configuração de Sessão**
- **Tratamento de Exceções**
- **Estratégias de Recuperação**
- **Integridade dos Dados**

A implementação robusta dessas práticas garante operações confiáveis mesmo durante interrupções temporárias dos serviços AWS.

Implementação de Operações Idempotentes

- **Definição Clara**
- **Chaves de Idempotência**
- **Proteção da Integridade**
- **Implementação Cuidadosa**

Garantem que operações repetidas não causem duplicação ou alterações inesperadas nos dados, essenciais para manter a consistência em cenários de retry, como no DynamoDB. Exigem planejamento e suporte adequado do serviço AWS.

Gerenciamento de Respostas HTTP

Interpretar e tratar códigos de status HTTP das APIs da AWS é essencial para lógicas de retry e fallback. Isso ajuda a diferenciar erros temporários de falhas críticas, melhorando a comunicação e a resiliência das integrações.



Resumo dos Pontos Principais

1. **Boto3 é Essencial**

   A biblioteca padrão para interagir com serviços AWS via Python.

2. **Configuração Segura**

   Gerenciamento adequado de credenciais e sessões é fundamental.

3. **Tratamento de Erros**

   Implemente estratégias robustas para lidar com exceções.

4. **Segurança em Primeiro Lugar**

   Use KMS para criptografia e siga as melhores práticas de segurança.